

Phase 1: Data Loading & Preprocessing - Technical Documentation

Project: Business Entity Resolution Challenge

Phase: 1 - Data Loading & Preprocessing

Date: September 25, 2026

Authors: Team Adityasengar18888

Table of Contents

- [Objective](#1-objective)
 - [Dataset Overview](#2-dataset-overview)
 - [Data Loading Strategy](#3-data-loading-strategy)
 - [Text Normalization Pipeline](#4-text-normalization-pipeline)
 - [Address Component Extraction](#5-address-component-extraction)
 - [Implementation Details](#6-implementation-details)
 - [Test Results](#7-test-results)
 - [Performance Benchmarks](#8-performance-benchmarks)
 - [Key Design Decisions](#9-key-design-decisions)
 - [Files Created](#10-files-created)
 - [Next Steps](#11-next-steps)
-

1. Objective

Build a robust preprocessing pipeline that normalizes noisy business records from 3 independent sources into a consistent, comparable format. This pipeline is the foundation for all downstream tasks (blocking, feature engineering, matching).

Key challenges addressed:

- Multi-script text (Devanagari, Tamil, Kannada, Latin, French accented)
- Extreme name variations (typos, abbreviations, URL-style names, DBA names)

- Inconsistent address formats across US, India, and France
- Large scale (~12.5 million records across train + test)

2. Dataset Overview

2.1 Dataset Scale

Split	Source 1 (Reference)	Source 2	Source 3	Total
Train	2,206,821	5,034,616	5,285,603	12,527,040
Test	1,732,544	4,887,273	5,082,316	11,702,133

2.2 Country Distribution

Training Set:

Country	Source 1	Source 2	Source 3
US	1,323,633	3,016,817	3,170,056
India	883,188	2,017,799	2,115,547

Test Set:

Country	Source 1	Source 2	Source 3
US	663,106	1,871,330	1,945,701
India	809,986	2,312,565	2,405,000
France (unseen)	259,452	703,378	731,615

2.3 Data Quality Issues

Issue	Source 1	Source 2	Source 3
Null business names	0	2	13
Null addresses	0	168,967 (3.4%)	175,916 (3.3%)
Non-Latin script names	Rare	Common (Hindi)	Common (Hindi, Tamil, Kannada)

2.4 Ground Truth Statistics

Metric	Value
Total S1 entities	2,206,821
Singletons (no matches)	123,247 (5.58%)
Average matches per entity	3.46
Max matches per entity	11

Match count distribution:

Matches	Count	Percentage
0 (singleton)	123,247	5.58%
1	119,157	5.40%
2	375,212	17.00%
3	530,841	24.05%
4	484,115	21.94%
5	321,957	14.59%
6	164,868	7.47%
7	63,968	2.90%
8+	23,456	1.06%

3. Data Loading Strategy

3.1 Loading Parameters

```
pd.read_csv(
    filepath,
    sep="\t",  # Tab-separated (addresses contain commas)
    dtype={"entity_id": str, "business_name": "string", "business_address": "string", "country": "category"},
    encoding="utf-8",  # Handle multi-script text
    na_values=["", "nan", "null", "None", "NaN"],
    keep_default_na=True,
)
```

Why these choices:

- `sep="\t"`: Mandatory - addresses and ID lists contain commas, so CSV parsing would break
- Memory-efficient dtypes: string (PyArrow backed in pandas ≥ 2.0) and category significantly reduce memory footprint for ~12.5M records

- dtype=str: Entity IDs like S1-965667 must not be parsed as integers
- encoding="utf-8": Required for Hindi (Devanagari), Tamil, Kannada scripts
- NaN values are filled with empty strings "" for safe string operations downstream

3.2 Memory Considerations

At ~12.5M records, the full dataset requires ~6-8 GB RAM. We:

- Use string dtype (not object) where possible
- Process by country partition when feasible
- Load the full dataset only when needed (sample-based development)

4. Text Normalization Pipeline

4.1 Business Name Normalization

The name normalization pipeline is applied in this exact order:

```
Raw Input
?
??? 1. Lowercase
??? 2. Transliterate non-Latin -> ASCII (unidecode)
??? 3. Strip accent marks (NFKD)
??? 4. Remove URL suffixes (.com, .org, .net, .in)
??? 5. Remove leading noise (-- prefixes)
??? 6. Clean punctuation (& -> and, hyphens -> space)
??? 7. Expand abbreviations (corp -> corporation, etc.)
?
?
Normalized Output
```

Step-by-Step Examples:

Example 1 - Accent Handling:

```

Input:  "Payne Énterprises"
Step 1: "payne énterprises"
Step 2: "payne Enterprises" (no non-Latin chars, skip)
Step 3: "payne enterprises" (strip accent from é)
Output: "payne enterprises"

```

Example 2 - URL-Style Name:

```

Input:  "maurewilliamscolombier.com"
Step 1: "maurewilliamscolombier.com"
Step 4: "maurewilliamscolombier" (.com removed)
Output: "maurewilliamscolombier"

```

Example 3 - Hindi Transliteration:

```

Input:  "???? ??? ???????? ????????"
Step 1: "???? ??? ???????? ????????"
Step 2: "eses phuudd praaivett limitedd" (unicode)
Output: "eses phuudd praaivett limitedd"

```

Example 4 - Leading Noise + Abbreviation:

```

Input:  "-- Holloway Peak Inc Seafood"
Step 1: "-- holloway peak inc seafood"
Step 5: "holloway peak inc seafood" (-- removed)
Step 7: "holloway peak incorporated seafood" (inc -> incorporated)
Output: "holloway peak incorporated seafood"

```

Example 5 - Legal Suffix Expansion:

```

Input:  "Pvt. EFS Print Ventures Ltd."
Step 1: "pvt. efs print ventures ltd."
Step 7: "private efs print ventures limited" (pvt->private, ltd->limited)
Output: "private efs print ventures limited"

```

4.2 Blocking Key Normalization

A more aggressive normalization that strips legal suffixes entirely. Used only for blocking (candidate generation), not for feature computation.

```
normalize_name_for_key("Raj Investments Private Limited") -> "raj investments"
normalize_name_for_key("Payne Enterprises LLC")             -> "payne"
normalize_name_for_key("Dahlia Power Reliable Scientific LLC") -> "dahlia power reliable scientific"
```

Legal suffixes removed: incorporated, corporation, company, limited, private, public, enterprises, enterprise, associates, association, holdings, group, solutions, services, technologies, technology, industries, international, consultants, consulting, ventures, partners, partnership, inc, corp, co, ltd, llc, llp, plc, pvt, sa, sas, sarl, gmbh, ag, dba, pllc, lp

4.3 Address Normalization

```
Raw Address
?
??? 1. Lowercase
??? 2. Transliterate non-Latin -> ASCII
??? 3. Strip accent marks
??? 4. Clean punctuation
??? 5. Expand address abbreviations
??? 6. Normalize state abbreviations -> full names
??? 7. Remove 'null' placeholders
?
?
Normalized Address
```

Examples:

Raw Address	Normalized
`3315 FREMONT ST, PEORIA, IL`	`3315 fremont street peoria illinois`
`KANSAS CITY, MO, 630 45ND TERRACE, null`	`kansas city missouri 630 45nd terrace`
`1795 Westchester Drive, High Point, NC`	`1795 westchester drive high point north carol..`
`AF-0684, NANDGRAM...GHAZIABAD, ?????? ??????`	`af 0684 nandgram...ghaziabad uttar pradesh`

Address abbreviations expanded (50+): st->street, rd->road, ave->avenue, blvd->boulevard, dr->drive, ln->lane, ct->court, hwy->highway, apt->apartment, ste->suite, n->north, s->south, e->east, w->west, and all US/India state abbreviations.

4.4 Transliteration Strategy

We use indic-transliteration for accurate conversion of Indian scripts (Devanagari, Tamil, Kannada) and fallback to unicode for French accents:

Script	Input	Output
Hindi (Devanagari)	??? ?????????	rAma mArkeTiMga
Tamil	???? ?????????????????	rAj innnvesDhmeNDhs
Kannada	???????	karnATaka
French accented	Énterprises	Enterprises

Methodology:

- Use indic_transliteration with sanscript.ITRANS for supported scripts
- Any remaining non-ASCII characters are handled via unicode

Limitation: Transliteration is approximate - it maps characters phonetically. This is acceptable because: 1. Both S1 (English) and S2/S3 (Hindi) records go through the same pipeline 2. The ML model in Phase 4 will learn to match across these variations using similarity features 3. For blocking, we use multiple keys so transliteration quality doesn't solely determine recall

5. Address Component Extraction

5.1 ZIP/PIN Code Extraction

Position-aware extraction to avoid false-matching street numbers:

Country	Pattern	Position Rule	Example
US	<code>`\b(\d{5})(?:-\d{4})?\b`</code>	Prefer last match	<code>`NC 27265` -> `27265`</code>
India	<code>`\b(\d{6})\b`</code>	Must not be first token	<code>`CHENNAI 600004` -> `600004`</code>
France	<code>`\b(\d{5})\b`</code>	Typically at start	<code>`75008 Paris` -> `75008`</code>

False-positive prevention:

```
"17560 Ellis Road, Tahlequah, OK" -> "" (17560 is a street number, not ZIP)
"105 ELM ST, MORGANTON, NC 28655" -> "28655" (actual ZIP at end)
```

5.2 Address Token Extraction

Extracts significant words from addresses for token-overlap blocking:

```
extract_address_tokens("3315 FREMONT ST, PEORIA, IL")
# -> ["fremont", "street", "peoria", "illinois"]
```

Stop words removed: near, behind, opposite, beside, next, to, the, of, at, in, on, and, or, no, number, floor, block, sector, phase, plot, kh, survey, post, office, box, po, null

5.3 Name Prefix

First 5 characters of the blocking-key name, for cheap prefix-based blocking:

```
"prabhav business center" -> "prabh"
"orelee s barbershop"      -> "orele"
"prime money"              -> "prime"
```

6. Implementation Details

6.1 Columns Added by Preprocessing

Column	Description	Example
`name_clean`	Fully normalized business name	`"payne enterprises"`
`name_key`	Blocking-key name (no legal suffixes)	`"payne"`
`addr_clean`	Fully normalized address	`"3315 fremont street peoria illinoi..`
`zip_pin`	Extracted ZIP/PIN code	`"28655"`
`name_pref..`	First 5 chars of name_key	`"payne"`
`addr_toke..`	Significant address tokens (space-joined)	`"fremont street peoria illinois"`
`has_name`	Boolean flag indicating if name is present	`True`
`has_addre..`	Boolean flag indicating if address is pres..	`True`

6.2 Ground Truth Utilities

Two utility functions for downstream use:

- `parse_ground_truth(gt_df) -> Dict[str, List[str]]`: Maps each S1 entity to its matched IDs
- `build_match_set(gt_dict) -> Set[Tuple[str, str]]`: Creates (S1, matched) pairs for O(1) lookup

7. Test Results

7.1 Unit Tests

All 30 tests across 6 categories passed:

Test Category	Tests	Status
Name Normalization	12/12	? PASS
Name Key (Blocking)	4/4	? PASS
Script Transliteration	4/4	? PASS
Address Normalization	5/5	? PASS
ZIP/PIN Extraction	5/5	? PASS
URL Removal	4/4	? PASS

7.2 Integration Test (5,000 rows per source)

Metric	S1	S2	S3
Records processed	5,000	5,000	5,000
ZIP/PIN found	334 (6.7%)	319 (6.4%)	273 (5.5%)
Empty names after norm	0	0	0
Empty addrs after norm	0	172	174
Avg normalized name length	25.5 chars	26.2 chars	26.1 chars
Avg address tokens	6.3	5.6	5.5

8. Performance Benchmarks

Processing speed on 5,000 records per source:

Operation	Speed (records/sec)
Name normalization	~90,000-100,000
Name key normalization	~83,000-91,000
Address normalization	~53,000-55,000
Total pipeline (all columns)	~40,000

Estimated full dataset processing time:

- Training set (12.5M records): ~5.2 minutes
 - Test set (11.7M records): ~4.9 minutes
 - Total: ~10 minutes
-

9. Key Design Decisions

9.1 Why Expand Abbreviations Instead of Standardizing to Short Form?

We expand "pvt" -> "private" rather than "private" -> "pvt" because: 1. Expanded forms have more character overlap for similarity metrics 2. Some abbreviations are ambiguous ("st" could be "street" or "saint") 3. Expanded forms produce better TF-IDF features

9.2 Why Keep Legal Suffixes in `name\_clean` But Remove in `name\_key`?

- name_clean (with suffixes): Used for feature computation - knowing that both records have "Private Limited" adds matching signal
- name_key (without suffixes): Used for blocking - legal suffixes are noise that can prevent blocking recall (e.g., "Raj Investments LLP" should block with "Raj Investments Private Limited")

9.3 Why Not Use Phonetic Encoding (Soundex/Metaphone) in Preprocessing?

Phonetic encoding is deferred to Phase 2 (Blocking) because:

- It's a blocking-specific technique, not a general normalization
- Different phonetic algorithms suit different use cases
- It adds complexity to the preprocessing output schema

9.4 Why Position-Aware ZIP Extraction?

Naive regex matching of `\d{5}` captures street numbers (e.g., 17560 in "17560 Ellis Road"). Our position-aware approach:

- For US: Only accepts 5-digit matches appearing >5 characters into the string
 - For India: Only accepts 6-digit matches not at position 0
 - Prefers the last match (ZIPs typically appear at the end of US addresses)
-

10. Files Created

File	Lines	Purpose
`code/business_entity_resolution/src/config.py`	155	Paths, abbreviation maps, hyperparam..
`code/business_entity_resolution/src/preprocess..	470	Full preprocessing pipeline
`code/business_entity_resolution/src/test_prepr..	220	30 unit tests
`code/business_entity_resolution/src/run_prepro..	98	Integration test on real data
`code/business_entity_resolution/src/__init__.py`	1	Package init
`code/business_entity_resolution/requirements.t..	8	Pinned dependencies

11. Next Steps

Phase 2: Blocking / Candidate Generation will: 1. Use country as a mandatory pre-filter (reduces comparisons ~50%) 2. Build multi-key blocking using: ZIP/PIN, name prefix, name TF-IDF top-K, address token overlap 3. Union all blocking keys per S1 entity to maximize recall 4. Target >98% blocking recall with ~1000:1 reduction ratio 5. Output: candidate_pairs.tsv with all plausible matches

Document generated as part of the Business Entity Resolution Challenge pipeline.